

Document number	P2227R0
Date	2020-10-02
Audience	LWG
Reply-to	Jonathan Wakely <cxx@kayari.org>

Update normative reference to POSIX

- - [Introduction](#)
 - [Motivation](#)
 - [Relation to the C standard](#)
 - [Undefined errno constants](#)
 - [File system operations](#)
 - [Regular expressions](#)
 - [Other mentions of POSIX](#)
 - [Proposed Changes](#)
 - [POSIX versions](#)
 - [Acknowledgments](#)
 - [References](#)

Introduction

The C++ standard has a normative reference to ISO/IEC 9945:2003 (aka POSIX.1-2001 aka The Single UNIX Specification, version 3). However, the C++ standard library refers to POSIX functions and macros which are not defined in that document, as they weren't added until ISO/IEC/IEEE 9945:2009 (aka POSIX.1-2008 aka SUSv4). The C++ standard should update its reference.

Motivation

When we refer to something in POSIX, we should ensure it is actually defined in the document we reference.

The index only contains three references to POSIX (for [intro.refs] and [re.matchflag]) which is woefully incomplete.

The relevant references to POSIX in the current C++ working paper are:

- [namespace.posix] refers to "ISO/IEC 9945".
- [errno] refers to <errno.h> and the Exxx constants. See below.
- [fs.conform.9945] refers to POSIX, but not to any specific features of it.
- [fs.class.path.general] has an example which refers to [4.11 Pathname Resolution](#).
- [fs.path.generic] refers to the [Portable Filename Character Set](#) which is unchanged.
- [fs.enum.perms] [tab:fs.enum.perms] refers to the [<sys/stat.h>](#) macros, which are unchanged.
- [fs.class.directory.iterator.general] refers to `readdir_r`.
- [fs.op.create.directory] "as if by POSIX `mkdir()`".
- [fs.op.create.dir.symlink] "as if by POSIX `symlink()`".
- [fs.op.create.hard.lk] "as if by POSIX `link()`".
- [fs.op.create.symlink] "as if by POSIX `symlink()`".
- [fs.op.current.path] "as if by POSIX `getcwd()`", "as if by POSIX `chdir()`".
- [fs.op.equivalent] "as if by the values of the POSIX `stat` class" (non-normative).
- [fs.op.file.size] "as if by POSIX `stat`".

- [fs.op.last.write.time] "as if by POSIX `stat()`", "as if by POSIX `futimens()`" (see below).
- [fs.op.permissions] "as if by POSIX `fchmodat()`" (see below).
- [fs.op.remove] "as if by POSIX `remove()`".
- [fs.op.remove.all] "as if by POSIX `remove()`".
- [fs.op.rename] "as if by POSIX `rename()`".
- [fs.op.resize.file] "as if by POSIX `truncate()`" (see below).
- [fs.op.space] "by using POSIX `statvfs` to obtain a POSIX struct `statvfs`" (see below).
- [fs.op.status] "as if by using POSIX `stat()`", "as if by POSIX `S_ISREG`" etc.
- [fs.op.symlink.status] "as if by using POSIX `lstat()` to obtain a POSIX struct `stat`", "as if by POSIX `S_ISLNK`".
- [re.matchflag] [tab:re.synopt] "See also: POSIX, Base Definitions and Headers, Section 9.3", "See also: POSIX, Base Definitions and Headers, Section 9.4".
- [re.grammar] "equivalent to that of the same features in regular expressions in POSIX".
- [thread.req.exception] "an `errno` of `EPERM` is reported by a POSIX API call", "since POSIX specifies an `errno` of `EPERM`".

Relation to the C standard

It is worth noting that POSIX.1-2001 and later editions refer to C99 by reference. C++17 refers to C11 and since C++20 we refer to C17. I don't think this is a problem.

We only ever define behaviour "as if by POSIX `foo`", not by actually requiring `foo` to exist in the C++ implementation, or for the C++ library implementation to actually use `foo` under the hood (which certainly isn't the case for the Microsoft Visual C++ library). As long as the behaviour of the C++ library functions is consistent with the specified behaviour of the POSIX function, it doesn't matter that the program is not executing in a C99 environment (or a C environment at all!)

The next revision of POSIX is likely to refer to C17, as C++20 does. If that is published before C++23 we can revisit this topic and in all probability reference the new POSIX standard. But I see no reason we can't make the important fixes proposed below without having the references to C be the same.

Undefined `errno` constants

In the original C++ standard the `<cerrno>` and `<errno.h>` headers contained only the macros required by ISO C, `EDOM`, `ERANGE` and `errno`. The C++11 standard expanded this and subclause [errno] now says:

The contents of the header `<cerrno>` are the same as the POSIX header `<errno.h>`, except that `errno` shall be defined as a macro.

Following this statement is a list of all the `Exxx` macros defined in the header. This list includes `ENOTRECOVERABLE` and `EOWNERDEAD`, but those macros are not defined in the document named in [intro.refs], see [POSIX.1-2001 `errno.h`](#). Those error numbers were added to POSIX in the 2008 revision, see [POSIX.1-2008 `errno.h`](#), which says:

The [ENOTRECOVERABLE] and [EOWNERDEAD] errors are added from The Open Group Technical Standard, 2006, Extended API Set Part 2.

The 2008 revision also allows `ENOTSUP` and `EOPNOTSUPP` to have the same value, which was not the case in POSIX.1-2001.

The POSIX-1.2008 standard also defines some additional error numbers which are not present in the C++ standard, `EDQUOT`, `EMULTIHOP`, and `ENOLINK`. POSIX doesn't give any meaning to these, it just says they are "Reserved". We should consider whether we want to reserve those in C++ too, as we already do for `ESTALE`, but I am not proposing that here.

File system operations

The C++17 standard incorporated the [File System TS](#), adding many new functions which are defined by reference to similar POSIX functions. The File System TS has a normative reference to POSIX, but uses an undated reference to ISO/IEC 9945 (which by ISO convention means the latest revision of that standard). When the contents of the TS were incorporated into the C++ IS we didn't ensure our reference to ISO/IEC 9945:2003 was still appropriate.

The reference to 4.11 from [fs.class.path.general] needs to be updated. It is [4.12 Pathname Resolution](#) in POSIX.1-2008 and [4.13 Pathname Resolution](#) in POSIX.1-2017 (which is ISO/IEC/IEEE 9945:2009 plus the 2013 and 2017 Technical Corrigenda). Since the TCs are just lists of changes, not a complete document, the 4.12 section number from ISO/IEC/IEEE 9945:2009 seems to be correct still.

Most of the POSIX functions we refer to are present in POSIX.1-2001, but the `futimens` functions used in [fs.ops.last.write.time] and the `fchmodat` function used in [fs.op.permissions] don't exist at all in POSIX.1-2001. Strictly speaking, this means the semantics of `filesystem::last_write_time` and `filesystem::permissions` are undefined.

Two other functions, (`truncate` and `statvfs`), are optional and not required to exist on conforming POSIX systems (they are part of the X/Open System Interfaces Extension (aka XSI)). In the POSIX.1-2008 revision those functions are moved from the XSI option to the Base specification, so they are no longer optional.

The `S_ISVTX` macro used with `stat` is an optional feature, part of XSI again. It remains optional even in POSIX.1-2008 and POSIX.1-2017.

There is also a non-normative note referencing `readdir_r` which is part of the Thread-Safe Functions (TSF) option in POSIX.1-2001, but moved to Base for POSIX.1-2008. However, the `readdir_r` API is flawed, deprecated by some implementations, and may be removed in a future version of POSIX. We should just refer to `readdir` in this note, as we are not dependent on anything specific to `readdir_r` here.

Regular expressions

The references in [re.grammar] are still the right sections for the latest POSIX standard, see [9.3 Basic Regular Expressions](#) and [9.4 Extended Regular Expressions](#).

Other mentions of POSIX

The subclause [value.error.codes] and several places in [filesystem] refer to POSIX-based operating systems, but that's a general description and not a specific reference to anything defined in POSIX. Those are not relevant here.

The subclause [support.runtime.general] refers to "the POSIX functions `setenv` and `getenv`", which needs no changes.

Subclause [format.string.std] has a note saying "This is similar to the semantics of the POSIX `wcswidth` function." That function is an XSI extension, so optional for POSIX implementations. But we aren't requiring it to be supported, only pointing out a similarity. The following paragraph has another use of "POSIX-based operating systems" which isn't relevant here.

Two footnotes in [locale.time.put.virtuals] refer to POSIX, but need no changes related to this paper (they shouldn't use "should" in notes, but that's another topic).

Proposed Changes

To fix these problems we need to change the normative reference in [intro.refs] to be a later revision of the POSIX standard, ISO/IEC/IEEE 9945:2009 including Corrigenda 1 (2013) and Corrigenda 2 (2017).

- Update [intro.refs] paragraph 1 as shown:

(1.6) — ISO/IEC/~~IEEE~~ 9945:~~2003~~2009, Information Technology — Portable Operating System Interface (POSIX)

(1.?) — [ISO/IEC/IEEE 9945:2009/Cor 1:2013, Information Technology — Portable Operating System Interface \(POSIX\), Technical Corrigendum 1](#)

(1.?) — [ISO/IEC/IEEE 9945:2009/Cor 2:2017, Information Technology — Portable Operating System Interface \(POSIX\), Technical Corrigendum 2](#)

- Update [namespace.posix] to add IEEE to the document number:

The namespace `posix` is reserved for use by ISO/IEC/~~IEEE~~ 9945 and other POSIX standards.

- Update [fs.class.path.general] paragraph 6 as shown:

Pathname resolution is the operating system dependent mechanism for resolving a pathname to a particular file in a file hierarchy. There may be multiple pathnames that resolve to the same file. *[Example 1: POSIX specifies the mechanism in section 4.~~11~~12, Pathname resolution. — end example]*

- Update [fs.class.directory.iterator.general] paragraph 11 as shown:

See POSIX `readdir`~~=f~~.

POSIX versions

Because the POSIX standard is republished by a number of different bodies it has different names and even publication years depending who you ask. See [POSIX Versions](#) and [Single UNIX Specification History](#) if you want to see what's what.

Acknowledgments

Thanks to Aaron Ballman and Eric Blake for comments on the initial draft.

References

ISO/IEC 14882:2017 Programming languages — C++ <https://www.iso.org/standard/68564.html>

ISO/IEC 9945-1:2003 Information technology — Portable Operating System Interface (POSIX) — Part 1: Base Definitions <https://www.iso.org/standard/38789.html>

ISO/IEC/IEEE 9945:2009 Information technology — Portable Operating System Interface (POSIX®) Base Specifications, Issue 7, <https://www.iso.org/standard/50516.html>

ISO/IEC/IEEE 9945:2009/Cor 1:2013 Information technology — Portable Operating System Interface (POSIX®) Base Specifications, Issue 7 — Technical Corrigendum 1, <https://www.iso.org/standard/62005.html>

ISO/IEC/IEEE 9945:2009/Cor 2:2017 Information technology — Portable Operating System Interface (POSIX®) Base Specifications, Issue 7 — Technical Corrigendum 2, <https://www.iso.org/standard/73314.html>

